

Ejercicios SQL – Caso Impocheck

Nivel 1

Insert/update/delete

- 1- Inserte 1 nuevo proveedor con 2 mails diferentes – utilice variables y SCOPE_IDENTITY
- 2- Inserte 2 nuevos artículos y sus precios en ambas listas– utilice variables y SCOPE_IDENTITY
- 3- Modifique el nombre del proveedor 5
- 4- Modifique el color (nuevo color “plata”) a todos los artículos que se venden en caja y su articuloID sea menor a 10
- 5- Modifique el límite de crédito de los distribuidores: 3010000011 / 3010000014 / 3010000019 / 3010000020 a 100mil pesos.
- 6- Inserte un nuevo empleado con 2 hijos que pertenezca al área de compras
- 7- Consulte los hijos (dni_hijo) que tiene el empleado 20000010
- 8- Elimine al empleado 20000010 y verifique la eliminación en cascada de sus hijos

Consultas

- 1-Un informe de las órdenes de compras anteriormente ingresadas al sistema con su fecha, proveedor, lista de ítems incluidos, cantidades de cada ítem, precio unitario, moneda de la OC, % de descuento si corresponde, la fecha de entrega pactada para cada ítem
- 2- Pedidos pendientes de aprobar: lista de pedidos pendientes de autorizar, indicando fecha de ingreso, cliente y monto total del pedido.
- 3- listado de facturas de proveedores pendientes de pago vencidas hasta la fecha y sus respectivos totales a pagar.

Nivel 2

Insert/update/deleted

- 1- Modifique el nombre del artículo que en la OC número 3 tuvo una compra por 12 unidades
- 2- Modifique el nombre y dirección del empleado que tomó la licencia tipo “vacacion” del 15 al 31 de marzo del 2026 (utilice un único update)

Consultas

1- Órdenes de Compra pendientes de aprobación

Listar todas las Órdenes de Compra (OC) creadas en el sistema que **todavía no fueron aprobadas** (es decir, no tienen un registro correspondiente en la tabla de aprobaciones).

2- Proveedores con desvíos en facturación

Listar los proveedores (id interno y nombre) que nos han facturado algún artículo a un precio diferente al acordado originalmente en la Orden de Compra.

3- Última compra proveedor

Un reporte de los proveedores a los que se les haya comprado con anterioridad ese producto detallando: ID interno de proveedor, nombre de la empresa, último precio unitario al que se compró el producto y la cantidad comprada en unidades

4- Pedidos pendientes de aprobar

Se requiere un reporte que indique para cada cliente (CUIT + nombre), su monto de límite de crédito asignado, el monto que tiene en pedidos aprobados y el monto que tiene en pedidos pendientes de aprobar.

5- Clientes con deudas vencidas que superen su límite de crédito

Listar los clientes (CUIT, razón social, límite de crédito) junto con su deuda total acumulada. Teniendo en cuenta que el cliente debe tener deuda vencida y no cobrada que supere su límite de crédito establecido.

Ejercicios para completar y ordenar

1- Completar la siguiente sentencia

```
SELECT
    d.CUIT AS CUIT,
    d.RazonSocial AS 'Razon social',
    d.NombreFantasia as '-----',
    d.email AS 'Correo Electronico',
    d.DireccionFiscal as 'direccion fiscal',
    d.ZonaCobertura as 'zona de Cobertura'-----
    d.limiteCredito as 'limite de crédito',
    d.condicionFiscal as 'condicion fiscal',
    d.fechaAlta as 'fecha de ingreso',
    ----- AS Sucursal,
    dir.direccion AS Direccion_Entrega,
    dir.telefono AS Telefono_Sucursal
    ----- Distribuidores d
LEFT JOIN DireccionesDistribuidores dir ----- d.CUIT = dir.CUIT
ORDER BY d.RazonSocial, dir.codigo_direccion;
```

2- Ordenar la siguiente sentencia

FROM Proveedores p

SELECT

p.Id_interno_Prov AS ID_Proveedor,

p.tax_id AS Tax_ID,

p.nombre_empresa AS Razon_Social,

p.pais AS Pais,

Sitio_Web AS p.sitio_web,

p.banco AS Banco,

p.CBU AS CBU_Cuenta,

AS Correo_Electronico, m.mail

m.tipo_contacto AS Tipo_Contacto

ORDER BY p.nombre_empresa, m.mail;

LEFT JOIN Proveedores_Mails m ON m.Id_interno_Prov = p.Id_interno_Prov

Soluciones propuestas (estas son solo algunas opciones, pueden haber otras)

Nivel 1

Insert/update/delete

- 1- Inserte 1 nuevo proveedor con 2 mails diferentes – utilice variables y SCOPE_IDENTITY

```
--nuevo proveedor
DECLARE @NuevoProvID INT;

-- 1. Insertamos el Proveedor
INSERT INTO Proveedores (tax_id, nombre_empresa, pais, sitio_web, id_moneda, banco, CBU)
VALUES (
    'TAX-20287-X',
    'Distribuciones Proter S.A.',
    'Argentina',
    '://distribucionesproter.com',
    1, -- Referencia a Pesos
    'Banco Galicia',
    '0170001234569' -- CBU de 13 dígitos
);

-- Capturamos el ID recién creado
SET @NuevoProvID = SCOPE_IDENTITY();

-- 2. Insertamos los 2 mails asociados
INSERT INTO Proveedores_Mails (Id_interno_Prov, mail, tipo_contacto)
VALUES
(@NuevoProvID, 'ventas_distribuciones@distribucionesproter.com', 'Comercial'),
(@NuevoProvID, 'administracion_admin@distribucionesproter.com', 'Facturación');
```

- 2- Inserte 2 nuevos artículos y sus precios en ambas listas – utilice variables y SCOPE_IDENTITY

```
-- Insertamos el nuevo artículo
INSERT INTO articulos (NumeroArticulo, Nombre, Descripcion, UnidadMedida,
VolumenM3, UnidadesPorBulto, PesoKilos, Color, CodigoEAN13, fechaIngreso)
VALUES (5004, 'Monitor Led 65', 'Monitor Full HD 65 pulgadas sonido all-around', 'Unidad',
0.0700, 3, 4.50, 'Negro', '7790000000052', GETDATE());

-- Capturamos el ID generado y le asignamos precio en la Lista 1
DECLARE @NuevoID INT = SCOPE_IDENTITY();
INSERT INTO articulos_Precios (Cod_Listas_Precios, ArtículoID, precio)
VALUES (1, @NuevoID, 716000.00),
(2, @NuevoID, 734500.00)
```

- 3- Modifique el nombre del proveedor 5

```
update Proveedores set nombre_empresa = 'Electronics SRL' WHERE Id_interno_Prov = 5
```

- 4- Modifique el color (nuevo color “plata”) a todos los artículos que se venden en caja y su articuloID sea menor a 10

```
update Articulos set Color = 'plata' where UnidadMedida = 'caja' and ArtículoID < 10
```

- 5- Modifique el límite de crédito de los distribuidores: 3010000011 / 3010000014 / 3010000019 / 3010000020 a 100mil pesos.

```
update Distribuidores set LimiteCredito = 100000.00 where CUIT = '3010000011' or
cuit = '3010000014' or cuit = '3010000019' or CUIT = '3010000020'
```

6- Inserte un nuevo empleado con 2 hijos que pertenezca al área de compras

```
INSERT INTO Personas (DNI, nombre, apellido, fecha_nacimiento, domicilio)
VALUES
(32000120, 'Antonella', 'Galapago', '1990-05-20', 'Av. Libertador 2423');

-- 2. Los damos de alta como Empleados (Referencia por DNI)
-- Usamos CUIL de 12 dígitos y Legajos únicos
INSERT INTO Empleados (DNI, fecha_ingreso, CUIL, num_legajo, id_area)
VALUES
(32000120, '2024-02-15', '233200012099', 5003, 4); -- Compras
-- 3. Les asignamos sus Hijos (DNI de hijos entre 6 y 9 dígitos)
INSERT INTO Hijo_Empleado (DNI_Empleado, DNI_Hijo)
VALUES
(32000120, 55000121), -- Hijo 1
(32000120, 58000289); -- Hijo 2
```

7- Consulte los hijos (dni_hijo) que tiene el empleado 20000010

```
select *
from Empleados inner join Hijo_Empleado
on empleados.DNI = Hijo_Empleado.DNI_Empleado
where Empleados.DNI = '20000010'
```

8- Elimine al empleado 20000010 y verifique la eliminación en cascada de sus hijos

```
begin tran -- ejecuto primero
delete from Empleados where DNI = '20000010' -- hago eliminacion
select *
from Hijo_Empleado
where DNI_Empleado = '20000010' -- verifico eliminacion
rollback -- vuelvo para atras la eliminación
```

Consultas

1-Un informe de las órdenes de compras anteriormente ingresadas al sistema con su fecha, proveedor, lista de ítems incluidos, cantidades de cada ítem, precio unitario, moneda de la OC, % de descuento si corresponde, la fecha de entrega pactada para cada ítem

```
SELECT compras.Numero_OC, compras.fecha_alta, compras.Id_interno_Prov,
Proveedores.nombre_empresa, Items_Compra.ArticuloID, Items_Compra.cantidad,
Items_Compra.precio, Compras.moneda_OC, Compras.porcentaje_descuento,
Items_Compra.fecha_entrega
FROM Compras
inner join Proveedores
on compras.Id_interno_Prov = Proveedores.Id_interno_Prov
inner join Items_Compra
on compras.Numero_OC = Items_Compra.Numero_OC
```

2- Pedidos pendientes de aprobar: lista de pedidos pendientes de autorizar, indicando fecha de ingreso, cliente y monto total del pedido.

```
-- pedidos pendientes de aprobar
SELECT
    PD.Id_pedido AS Numero_Pedido,
    PD.fecha_alta AS Fecha_Ingreso,
    d.RazonSocial AS Cliente,
    (IPP.cantidad_pedida * arp.precio) AS 'Monto total'
FROM Pedidos_Distribuidores PD
INNER JOIN Distribuidores d ON PD.CUIT_Cliente = d.CUIT
INNER JOIN Items_Pedido IPP ON PD.Id_pedido = IPP.Id_pedido
inner join Articulos_Precios ARP on pd.Cod_Listas_Precios = arp.Cod_Listas_Precios
    and IPP.ArticuloID = ARP.ArticuloID
WHERE NOT EXISTS (
    -- Filtro dinámico: Excluye los pedidos que ya fueron autorizados
    SELECT 1
    FROM PedidosAprobados pa
    WHERE pa.Id_pedido = pd.Id_pedido)
ORDER BY pd.fecha_alta ASC; -- Ordenado por antigüedad para priorizar atención
```

3- listado de facturas de proveedores pendientes de pago vencidas a la fecha y sus respectivos totales a pagar.

```
declare @FechaDesde date = '2026-01-01'
declare @fechaHasta date = getdate()
SELECT
    f.id_remito AS Numero_Factura,
    f.referencia_proveedor AS Ref_Comprobante,
    p.nombre_empresa AS Proveedor,
    f.fecha_alta AS Fecha_Emission,
    f.fecha_vencimiento AS Fecha_Vencimiento,
    -- Sumamos el neto + impuestos para obtener el total a pagar
    (f.monto_sin_impuestos + f.monto_impuestos) AS Total_a_Pagar
FROM Facturas_Proveedores f
INNER JOIN Remitos_Compras r ON f.id_remito = r.id_remito
INNER JOIN Compras c ON r.Numero_OC = c.Numero_OC
INNER JOIN Proveedores p ON c.Id_interno_Prov = p.Id_interno_Prov
WHERE
    -- 1. Filtro por el vencimiento
    f.fecha_vencimiento BETWEEN @FechaDesde AND @fechaHasta

    -- 2. Filtro de exclusión: Solo las que NO estén en la tabla de pagos
    AND NOT EXISTS (
        SELECT 1
        FROM Facturas_a_pagar fp
        WHERE fp.id_remito = f.id_remito
    )
ORDER BY f.fecha_vencimiento ASC;
```

Nivel 2

Insert/update/delete

- 1- Modifique el nombre del articulo que en la OC número 3 tuvo una compra por 12 unidades

```
update Articulos set Nombre = 'teclado portatil de silicona'
where ArtículoID =
(select ic.ArtículoID from Items_Compra ic inner join Compras
on ic.Numero_OC = compras.Numero_OC
where Compras.Numero_OC = 3 and ic.cantidad = 12)
```

- 2- Modifique el nombre y dirección del empleado que tomó la licencia tipo “vacaciones” del 15 al 31 de marzo del 2026 (utilice un único update)

```
update Personas set personas.nombre = 'Juan Martin' , personas.domicilio = 'Sanchez de bustamante 986'
where personas.dni = (
select Empleados.DNI
from Empleados
inner join Licencias_empleados on Empleados.DNI = Licencias_empleados.DNI_empleado
inner join Tipos_Licencias on Licencias_empleados.id_tipo_licencia = Tipos_Licencias.id_tipo_licencia
where Licencias_empleados.fecha_inicio = '2026-03-15' and
Licencias_empleados.fecha_fin = '2026-03-31' and
Tipos_Licencias.nombre_licencia like 'vacacion%')
```

Consultas

1- Órdenes de Compra pendientes de aprobación (NOT EXISTS)

Queremos listar todas las Órdenes de Compra (OC) creadas en el sistema que **todavía no fueron aprobadas** (es decir, no tienen un registro correspondiente en la tabla de aprobaciones).

```
SELECT Numero_OC, fecha_alta, Id_interno_Prov
FROM Compras c
WHERE NOT EXISTS (
    SELECT 1
    FROM Compras_Aprobadas ca
    WHERE ca.Numero_OC = c.Numero_OC);

-- esta otra opcion es igualmente valida

select c.Numero_OC, c.fecha_alta, c.Id_interno_Prov
from Compras c
except
select ca.Numero_OC, compras.fecha_alta, compras.Id_interno_Prov
from Compras_Aprobadas ca
inner join Compras on ca.Numero_OC = compras.Numero_OC
```

2- Proveedores con desvíos en facturación (= ANY)

Listar los proveedores (id interno y nombre) que nos han facturado algún artículo a un precio diferente al acordado originalmente en la Orden de Compra.

```

SELECT Id_interno_Prov, nombre_empresa
FROM Proveedores
WHERE Id_interno_Prov = ANY (
    SELECT c.Id_interno_Prov
    FROM Items_Remitidos_Facturados irf
    INNER JOIN Compras c ON irf.Numero_OC = c.Numero_OC
    INNER JOIN Items_Compra ic ON irf.Numero_OC = ic.Numero_OC AND irf.ArticuloID = ic.ArticuloID
    WHERE irf.precio_unitario_final <> ic.precio );

```

3- Ultima compra proveedor

Un reporte de los proveedores a los que se les haya comprado con anterioridad ese producto detallando: ID interno de proveedor, nombre de la empresa, último precio unitario al que se compró el producto y la cantidad comprada en unidades

```

SELECT
    p.Id_interno_Prov,
    p.nombre_empresa,
    c.Numero_OC AS Ultima_OC,
    c.fecha_alta AS Fecha_Compra,
    a.Nombre AS Articulo,
    ic.cantidad,
    ic.precio,
    (ic.cantidad * ic.precio) AS Subtotal_Item
FROM Proveedores p
INNER JOIN Compras c ON p.Id_interno_Prov = c.Id_interno_Prov
-- Bajamos al detalle de los artículos de esa orden específica
INNER JOIN Items_Compra ic ON c.Numero_OC = ic.Numero_OC
INNER JOIN articulos a ON ic.ArticuloID = a.ArticuloID
-- Unimos con la subconsulta para quedarnos SOLO con la última OC de cada uno
right join (SELECT
    Id_interno_Prov,
    MAX(Numero_OC) AS Ultimo_Numero_OC
FROM Compras
GROUP BY Id_interno_Prov ) as UltimaCompraPorProveedor
on c.Numero_OC = UltimaCompraPorProveedor.Ultimo_Numero_OC
ORDER BY p.Id_interno_Prov, a.Nombre;

```

Segunda opción – utilización del “with”


```

-- Consulta de la última compra con detalle de items
WITH UltimaCompraPorProveedor AS (
  -- Esta subconsulta encuentra el ID de la última OC de cada proveedor
  SELECT
    Id_interno_Prov,
    MAX(Numero_OC) AS Ultimo_Numero_OC
  FROM Compras
  GROUP BY Id_interno_Prov )
SELECT
  p.Id_interno_Prov,
  p.nombre_empresa,
  c.Numero_OC AS Ultima_OC,
  c.fecha_alta AS Fecha_Compra,
  a.Nombre AS Artículo,
  ic.cantidad,
  ic.precio,
  (ic.cantidad * ic.precio) AS Subtotal_Item
FROM Proveedores p
-- Unimos con la subconsulta para quedarnos SOLO con la última OC de cada uno
INNER JOIN UltimaCompraPorProveedor ucc ON p.Id_interno_Prov = ucc.Id_interno_Prov
INNER JOIN Compras c ON ucc.Ultimo_Numero_OC = c.Numero_OC
-- Bajamos al detalle de los artículos de esa orden específica
INNER JOIN Items_Compra ic ON c.Numero_OC = ic.Numero_OC
INNER JOIN articulos a ON ic.ArticuloID = a.ArticuloID
ORDER BY p.Id_interno_Prov, a.Nombre;

```

4- Pedidos pendientes de aprobar

Se requiere un reporte que indique para cada cliente (CUIT + nombre), su monto de límite de crédito asignado, el monto que tiene en pedidos aprobados y el monto que tiene en pedidos pendientes de aprobar.

```

SELECT
  d.CUIT,
  d.RazonSocial AS Nombre_Cliente,
  d.limiteCredito AS Limite_Credito_Asignado,
  -- Subconsulta 1: Total en Pedidos APROBADOS
  (
    SELECT SUM(ip.cantidad_pedida * ap.precio)
    FROM Pedidos_Distribuidores p
    INNER JOIN Items_Pedido ip ON p.Id_pedido = ip.Id_pedido
    INNER JOIN articulos_Precios ap ON p.Cod_Listas_Precios = ap.Cod_Listas_Precios
    AND ip.ArticuloID = ap.ArticuloID
    WHERE p.CUIT_Cliente = d.CUIT
    -- Condición externa: El pedido DEBE existir en la tabla de aprobados
    AND EXISTS ( SELECT 1
      FROM PedidosAprobados pa
      WHERE pa.Id_pedido = p.Id_pedido ) )AS Monto_Pedidos_Aprobados,
  -- Subconsulta 2: Total en Pedidos PENDIENTES DE APROBAR
  (
    SELECT SUM(ip.cantidad_pedida * ap.precio)
    FROM Pedidos_Distribuidores p
    INNER JOIN Items_Pedido ip ON p.Id_pedido = ip.Id_pedido
    INNER JOIN articulos_Precios ap ON p.Cod_Listas_Precios = ap.Cod_Listas_Precios
    AND ip.ArticuloID = ap.ArticuloID
    WHERE p.CUIT_Cliente = d.CUIT
    -- Condición externa: El pedido NO debe existir en la tabla de aprobados
    AND NOT EXISTS (
      SELECT 1
      FROM PedidosAprobados pa
      WHERE pa.Id_pedido = p.Id_pedido ) )AS Monto_Pedidos_Pendientes
FROM Distribuidores d
ORDER BY d.RazonSocial;

```

Utilización de COALESCE PARA EVITAR LOS NULLS: La función COALESCE(..., 0) asegura que si un cliente es nuevo o simplemente no tiene

pedidos en alguno de los dos estados, la columna muestre \$0.00 de forma limpia en lugar de un espacio vacío (NULL).

```
SELECT
    d.CUIT,
    d.RazonSocial AS Nombre_Cliente,
    d.limiteCredito AS Limite_Credito_Asignado,
    -- Subconsulta 1: Total en Pedidos APROBADOS
    COALESCE((
        SELECT SUM(ip.cantidad_pedida * ap.precio)
        FROM Pedidos_Distribuidores p
        INNER JOIN Items_Pedido ip ON p.Id_pedido = ip.Id_pedido
        INNER JOIN articulos_Precios ap ON p.Cod_Listas_Precios = ap.Cod_Listas_Precios
        AND ip.ArticuloID = ap.ArticuloID
        WHERE p.CUIT_Cliente = d.CUIT
        -- Condición externa: El pedido DEBE existir en la tabla de aprobados
        AND EXISTS ( SELECT 1
            FROM PedidosAprobados pa
            WHERE pa.Id_pedido = p.Id_pedido ) ), 0) AS Monto_Pedidos_Aprobados,
    -- Subconsulta 2: Total en Pedidos PENDIENTES DE APROBAR
    COALESCE((
        SELECT SUM(ip.cantidad_pedida * ap.precio)
        FROM Pedidos_Distribuidores p
        INNER JOIN Items_Pedido ip ON p.Id_pedido = ip.Id_pedido
        INNER JOIN articulos_Precios ap ON p.Cod_Listas_Precios = ap.Cod_Listas_Precios
        AND ip.ArticuloID = ap.ArticuloID
        WHERE p.CUIT_Cliente = d.CUIT
        -- Condición externa: El pedido NO debe existir en la tabla de aprobados
        AND NOT EXISTS (
            SELECT 1
            FROM PedidosAprobados pa
            WHERE pa.Id_pedido = p.Id_pedido ) ), 0) AS Monto_Pedidos_Pendientes
FROM Distribuidores d
ORDER BY d.RazonSocial;
```

5- Clientes con deudas vencidas que superen su limite de crédito.

Listar los clientes (CUIT, razón social, limite de crédito) junto con su deuda total acumulada. Teniendo en cuenta que el cliente debe tener deuda vencida y no cobrada que supere su límite de crédito establecido.

Para armar este reporte complejo que cruza **clientes, facturas impagas y límites de crédito**, utilizaremos un enfoque combinado:

1. **EXISTS**: Para filtrar únicamente a los clientes que **tienen facturas con cuotas vencidas** hoy, basándonos en la tabla de vencimientos.
2. **ANY**: Para comparar si la sumatoria de su deuda actual acumulada supera su límite de crédito configurado.

```

SELECT
d.CUIT,
d.RazonSocial,
d.limiteCredito,
-- Subconsulta para calcular la deuda total actual del cliente
(SELECT SUM(v.monto_cuota)
FROM Facturas_Ventas_Vencimientos v
INNER JOIN Remitos_Ventas r ON v.id_remito_Venta = r.id_remito_Venta
WHERE r.CUIT_Cliente = d.CUIT
-- Filtramos que la cuota no haya sido cobrada aún
AND NOT EXISTS (
SELECT 1
FROM Cuotas_Cobradas cc
WHERE cc.id_remito_Venta = v.id_remito_Venta
AND cc.numero_cuota = v.numero_cuota ) ) AS Deuda_Total_Acumulada
FROM Distribuidores d
WHERE
-- 1. CONTROL DE EXISTENCIA: El cliente debe tener al menos una cuota VENCIDA hoy
-- 1. CONTROL DE VALOR (ANY): Su deuda total calculada supera su límite de crédito
AND d.limiteCredito < ANY (
SELECT SUM(fvv2.monto_cuota)
FROM Facturas_Ventas_Vencimientos fvv2
INNER JOIN Remitos_Ventas rv2 ON fvv2.id_remito_Venta = rv2.id_remito_Venta
WHERE rv2.CUIT_Cliente = d.CUIT
AND NOT EXISTS (
SELECT 1
FROM Cuotas_Cobradas cc2
WHERE cc2.id_remito_Venta = fvv2.id_remito_Venta
AND cc2.numero_cuota = fvv2.numero_cuota )
GROUP BY rv2.CUIT_Cliente );

```

¿Cómo procesa el motor esta consulta?

- **Paso 1 (EXISTS):** El sistema escanea la tabla de distribuidores uno por uno y descarta inmediatamente a los que están al día. Solo retiene a aquellos que tienen alarmas financieras por cuotas impagas expiradas.
- **Paso 2 (ANY):** Para los que quedaron seleccionados, calcula su deuda total latente (vencida y por vencer). Con el operador < ANY, evalúa si el limiteCredito del distribuidor quedó por debajo de esa bolsa de deuda. Si se cumple, el cliente entra en el listado.

Ejercicios para completar y ordenar

- 1- Consulta de datos de distribuidores

--listado de distribuidores y sus direcciones

```
SELECT
  d.CUIT AS CUIT,
  d.RazonSocial AS 'Razon social',
  d.NombreFantasia as 'Nombre de fantasia',
  d.email AS 'Correo Electronico',
  d.DireccionFiscal as 'direccion fiscal',
  d.ZonaCobertura as 'zona de Cobertura',
  d.limiteCredito as 'limite de crédito',
  d.condicionFiscal as 'condicion fiscal',
  d.fechaAlta as 'fecha de ingreso',
  dir.codigo_direccion AS Sucursal,
  dir.direccion AS Direccion_Entrega,
  dir.telefono AS Telefono_Sucursal
FROM Distribuidores d
LEFT JOIN DireccionesDistribuidores dir ON d.CUIT = dir.CUIT
ORDER BY d.RazonSocial, dir.codigo_direccion;
```

2- Orden de consulta de proveedores

--listado de proveedores

```
SELECT
  p.Id_interno_Prov AS ID_Proveedor,
  p.tax_id AS Tax_ID,
  p.nombre_empresa AS Razon_Social,
  p.pais AS Pais,
  p.sitio_web AS Sitio_Web,
  p.banco AS Banco,
  p.CBU AS CBU_Cuenta,
  m.mail AS Correo_Electronico,
  m.tipo_contacto AS Tipo_Contacto
FROM Proveedores p
LEFT JOIN Proveedores_Mails m ON p.Id_interno_Prov = m.Id_interno_Prov
ORDER BY p.nombre_empresa, m.mail;
```